



Automation

Jenkins Fundamentals



Outline

1

Pendahuluan

2

Instalasi & Konfigurasi Jenkins

3

Membuat dan Menjalankan Pipeline

1

Pendahuluan

Jenkins Fundamentals



Apa itu **CI/CD**?

Continuous Integration (CI)

Praktik pengembangan perangkat lunak di mana anggota tim mengintegrasikan kode mereka secara teratur, biasanya beberapa kali sehari. Setiap integrasi diverifikasi oleh build otomatis untuk mendeteksi kesalahan secepat mungkin.

Continuous Delivery/Deployment (CD)

Proses otomatis untuk menyiapkan kode yang telah lulus semua tahapan build dan test agar siap dirilis ke produksi. Continuous Deployment melangkah lebih jauh dengan otomatis menerapkan setiap perubahan yang lulus semua tahapan ke lingkungan produksi.



Manfaat Implementasi CI/CD



Pengurangan Bug

Integrasi dan pengujian yang sering membantu menemukan dan memperbaiki bug lebih awal dalam siklus pengembangan, mengurangi biaya perbaikan.



Rilis Lebih Cepat

Otomatisasi proses build, test, dan deployment memungkinkan tim untuk merilis fitur baru dengan lebih cepat dan lebih sering ke pengguna.



Peningkatan Kualitas Kode

Pengujian otomatis dan praktik code review yang konsisten membantu mempertahankan standar kode yang tinggi di seluruh proyek.

Peran Jenkins dalam **Ekosistem CI/CD**

Integrasi Kode

Jenkins mengotomatisasi proses pengambilan kode dari repositori dan membangun aplikasi.



Pengujian

Menjalankan berbagai jenis pengujian untuk memastikan kualitas kode.



Jenkins

Deployment

Mengirimkan aplikasi ke lingkungan target secara otomatis.



Pengemasan

Menghasilkan artefak yang siap untuk deployment.



Pengenalan Jenkins

Sejarah Singkat

Jenkins awalnya dikembangkan sebagai Hudson oleh Kohsuke Kawaguchi di Sun Microsystems pada 2004. Setelah Oracle mengakuisisi Sun, proyek ini bercabang menjadi Jenkins pada 2011 dan terus berkembang sebagai proyek open source yang populer.

Fitur Utama

- Sistem plugin yang ekstensif (lebih dari 1500 plugin)
- Pipeline as Code untuk mendefinisikan alur CI/CD
- Dukungan untuk build terdistribusi
- Integrasi dengan berbagai SCM, build tools, dan cloud providers

Arsitektur Jenkins

Arsitektur Jenkins dirancang untuk skalabilitas dan fleksibilitas. Jenkins Master bertanggung jawab untuk menerima permintaan build, mendistribusikan pekerjaan ke agen, dan menyajikan antarmuka pengguna. Agen Jenkins menjalankan tugas yang didelegasikan oleh master, memungkinkan distribusi beban kerja di beberapa mesin.

Jenkins Master

Mengelola konfigurasi, antarmuka pengguna, dan penjadwalan

Komunikasi Master-Agent

Protokol komunikasi untuk mendistribusikan tugas

Jenkins Agents/Slaves

Menjalankan tugas build yang didelegasikan oleh master



2

Instalasi & Konfigurasi Jenkins

Jenkins Fundamentals

Prasyarat Instalasi Jenkins



Java Development Kit (JDK)

- JDK 8, 11, atau 17
- Variabel lingkungan JAVA_HOME harus dikonfigurasi
- `java -version` untuk memverifikasi instalasi



Spesifikasi Hardware Minimal

- 256MB RAM (disarankan 1GB+)
- 1GB ruang disk (disarankan 10GB+)
- Prosesor dual-core untuk performa optimal



Jaringan

- Port default 8080 untuk antarmuka web
- Port 50000 untuk komunikasi agen
- Akses internet untuk mengunduh plugin

Metode Instalasi Jenkins: Docker

Instalasi Docker

Pastikan Docker terinstal di sistem Anda. Verifikasi dengan perintah: `docker --version`

Tarik Image Jenkins

Jalankan perintah: `docker pull jenkins/jenkins:its`

Jalankan Container Jenkins

`docker run -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home jenkins/jenkins:its`

Akses Jenkins

Buka browser dan navigasi ke <http://localhost:8080>

Metode Instalasi Jenkins: **War File**

Unduh Jenkins WAR

Kunjungi jenkins.io/download dan unduh file jenkins.war terbaru. File WAR (Web Application Archive) berisi semua yang diperlukan untuk menjalankan Jenkins.

Jalankan Jenkins

Buka terminal atau command prompt, navigasi ke direktori tempat file jenkins.war berada, dan jalankan perintah: `java -jar jenkins.war`

Akses Jenkins

Setelah proses startup selesai, buka browser web dan navigasi ke `http://localhost:8080` untuk mengakses antarmuka pengguna Jenkins.

Konfigurasi Awal Jenkins

Setelah mengakses Jenkins untuk pertama kali, Anda akan melalui proses konfigurasi awal yang meliputi: membuka password admin awal dari file system, memilih plugin yang akan diinstal (disarankan atau kustom), membuat pengguna admin pertama, dan mengkonfigurasi URL Jenkins.



Membuka Password Admin Awal

Lokasi Password

Password admin awal disimpan dalam file `initialAdminPassword` di direktori Jenkins. Lokasi file ini akan ditampilkan di layar setup.

- Untuk instalasi Docker: Lihat log container dengan `docker logs [container_id]`
- Untuk instalasi WAR: Lihat output konsol atau file di lokasi yang ditampilkan

Password ini hanya digunakan untuk setup awal dan dapat diganti setelah Anda membuat pengguna admin permanen. Pastikan untuk menyimpan password ini dengan aman hingga proses konfigurasi selesai.



Instalasi Plugin Dasar



Suggested Plugins

Opsi ini menginstal set plugin yang umumnya digunakan, termasuk integrasi Git, pipeline, dan alat build seperti Maven. Ini adalah pilihan terbaik untuk pemula yang ingin memulai dengan cepat.



Select Plugins

Opsi ini memungkinkan Anda memilih plugin spesifik yang ingin diinstal. Berguna jika Anda memiliki kebutuhan khusus atau ingin instalasi Jenkins yang minimal.



Plugin Manager

Setelah instalasi awal, Anda dapat menambah atau menghapus plugin melalui Plugin Manager di menu Manage Jenkins.

Membuat User Admin Pertama

Membuat pengguna admin pertama sangat penting untuk keamanan instalasi Jenkins Anda. Pengguna ini akan memiliki akses penuh ke semua fitur dan konfigurasi Jenkins. Pastikan untuk menggunakan password yang kuat dan menyimpan informasi login dengan aman.

Isi Formulir Pengguna

Masukkan username, password, nama lengkap, dan email

Tetapkan Izin

Admin pertama otomatis mendapatkan semua izin

Simpan Kredensial

Catat informasi login di tempat yang aman

URL Konfigurasi Jenkins

URL Jenkins

Selama setup, Anda akan diminta untuk mengkonfirmasi URL Jenkins. Ini adalah alamat yang akan digunakan untuk mengakses Jenkins dari browser.

- Untuk penggunaan lokal: `http://localhost:8080`
- Untuk server: `http://[nama-server-atau-ip]:8080`
- Dengan reverse proxy: `https://jenkins.example.com`

URL ini penting untuk berbagai fitur Jenkins, termasuk notifikasi email dan webhook. Pastikan URL yang dikonfigurasi dapat diakses oleh semua sistem yang perlu berkomunikasi dengan Jenkins.



Manajemen Plugin

Pentingnya Plugin

Plugin memperluas fungsionalitas Jenkins, memungkinkan integrasi dengan berbagai alat dan platform. Ekosistem plugin yang kaya adalah salah satu kekuatan utama Jenkins.

Mencari Plugin

Akses Manage Jenkins -> Manage Plugins -> Available tab. Gunakan kotak pencarian untuk menemukan plugin berdasarkan nama atau fungsi.

Menginstal Plugin

Centang plugin yang diinginkan dan klik "Install without restart" atau "Download now and install after restart" tergantung pada kebutuhan Anda.

Plugin Penting untuk Jenkins

Git Integration

Memungkinkan Jenkins untuk mengambil kode dari repositori Git dan memicu build berdasarkan perubahan kode.

Pipeline

Mendukung implementasi Pipeline as Code untuk mendefinisikan alur CI/CD dalam bentuk kode.

Maven Integration

Menyediakan integrasi dengan Apache Maven untuk build proyek Java.

Docker

Memungkinkan Jenkins untuk membangun dan menjalankan container Docker sebagai bagian dari pipeline.

3

Membuat dan Menjalankan Pipeline

Jenkins Fundamentals



Konsep CI/CD dan Pipeline

Review Singkat CI/CD

CI/CD adalah praktik DevOps yang menggabungkan integrasi berkelanjutan (CI) dan pengiriman/penerapan berkelanjutan (CD) untuk mengotomatisasi siklus hidup pengembangan perangkat lunak.

Tujuannya adalah untuk memungkinkan pengembang mengintegrasikan kode secara teratur, mengujinya secara otomatis, dan mengirimkannya ke produksi dengan cepat dan andal.

Jenkins Pipeline

Jenkins Pipeline adalah suite plugin yang mendukung implementasi dan integrasi pipeline continuous delivery di Jenkins. Pipeline didefinisikan sebagai kode, biasanya dalam file Jenkinsfile yang disimpan di repositori kode.

Pipeline as Code memungkinkan versioning, reusability, dan auditability dari proses CI/CD Anda.

Manfaat **Pipeline as Code**

Versioning

Pipeline didefinisikan dalam file yang dapat disimpan dalam version control, memungkinkan pelacakan perubahan dan rollback jika diperlukan.

Reusability

Pipeline dapat digunakan kembali di beberapa proyek, mempromosikan konsistensi dan mengurangi duplikasi.

Auditability

Perubahan pada pipeline dapat dilacak dan diaudit, meningkatkan transparansi dan akuntabilitas.

Testability

Pipeline dapat diuji seperti kode lainnya, memastikan bahwa perubahan tidak merusak proses CI/CD.

Jenis Pipeline di Jenkins

Freestyle Project

Jenis proyek Jenkins tradisional yang dikonfigurasi melalui antarmuka pengguna. Cocok untuk tugas sederhana dan pengguna yang lebih suka pendekatan berbasis GUI.

- Konfigurasi melalui UI
- Mudah untuk pemula
- Terbatas dalam kompleksitas
- Sulit untuk versi dan berbagi

Pipeline Project

Mendefinisikan alur CI/CD sebagai kode dalam file Jenkinsfile. Menawarkan fleksibilitas dan kemampuan untuk menangani alur kerja kompleks.

- Didefinisikan sebagai kode
- Dapat disimpan dalam version control
- Mendukung alur kerja kompleks
- Dua sintaks: Declarative dan Scripted

Declarative vs Scripted Pipeline

Declarative Pipeline

- Sintaks yang lebih terstruktur dan mudah dibaca
- Menggunakan blok pipeline { } sebagai blok tingkat atas
- Memiliki struktur yang telah ditentukan
- Lebih mudah dipelajari untuk pemula
- Disarankan untuk sebagian besar kasus penggunaan

Scripted Pipeline

- Berbasis Groovy scripting
- Menggunakan blok node { } sebagai blok tingkat atas
- Lebih fleksibel dan kuat
- Memerlukan pengetahuan Groovy
- Cocok untuk kasus penggunaan kompleks

Sintaks Dasar Declarative Pipeline

```
pipeline { agent any stages { stage('Build') { steps {  
    { echo 'Building..' sh 'mvn compile' } }  
stage('Test') { steps { echo 'Testing..' sh  
'mvn test' } } stage('Deploy') { steps {  
echo 'Deploying....' } } } }
```

Struktur Utama

- pipeline: Blok tingkat atas yang berisi seluruh pipeline
- agent: Menentukan di mana pipeline akan dijalankan
- stages: Berisi semua tahapan pipeline
- stage: Mendefinisikan satu tahap dalam pipeline
- steps: Tindakan yang akan dilakukan dalam tahap



Penjelasan Agent dalam Pipeline

agent any

Pipeline akan dijalankan pada agen Jenkins apa pun yang tersedia. Ini adalah opsi paling sederhana dan umum digunakan.

agent none

Tidak ada agen global yang ditentukan. Agen harus ditentukan untuk setiap tahap. Berguna untuk pipeline kompleks dengan kebutuhan lingkungan berbeda per tahap.

agent { label 'my-label' }

Pipeline akan dijalankan pada agen dengan label tertentu. Berguna untuk menjalankan build pada mesin dengan konfigurasi spesifik.

agent { docker 'maven:3.8.1-openjdk-11' }

Pipeline akan dijalankan dalam container Docker. Menyediakan lingkungan yang konsisten dan terisolasi untuk build.

Penjelasan Stages dan Stage

Stages adalah bagian utama dari pipeline Jenkins yang mewakili fase berbeda dalam proses CI/CD. Setiap stage berisi satu set steps yang akan dieksekusi secara berurutan. Stages memberikan visualisasi yang jelas tentang kemajuan pipeline dan memungkinkan paralelisasi jika diperlukan.



Build

Mengkompilasi kode sumber dan menghasilkan artefak.



Test

Menjalankan unit test, integration test, dan test lainnya.



Package

Mengemas aplikasi ke dalam format yang dapat didistribusikan.



Deploy

Mengirim aplikasi ke lingkungan target.

Penjelasan Steps dalam Pipeline



Perintah Shell

- sh 'command': Menjalankan perintah shell di lingkungan Unix/Linux
- bat 'command': Menjalankan perintah batch di lingkungan Windows
- powershell 'command': Menjalankan perintah PowerShell di Windows



Tools dan Utilitas

- echo 'message': Mencetak pesan ke log build
- git 'url': Mengkloning repositori Git
- checkout scm: Mengambil kode dari SCM yang dikonfigurasi
- archiveArtifacts 'pattern': Mengarsipkan artefak build



Kontrol Alur

- when { condition }: Menjalankan stage hanya jika kondisi terpenuhi
- parallel { tasks }: Menjalankan tugas secara paralel
- timeout(time: 1, unit: 'HOURS'): Menetapkan batas waktu

Contoh Jenkinsfile Sederhana

```
pipeline { agent any stages { stage('Hello') {  
steps { echo 'Hello, Jenkins!' } }  
stage('Build') { steps { echo 'Building the  
application...' sh 'mkdir -p build' sh 'touch  
build/empty.txt' } } stage('Test') { steps  
{ echo 'Running tests...' sh 'test -f  
build/empty.txt' } } stage('Deploy') {  
steps { echo 'Deploying the application...' } }  
} post { success { echo 'Pipeline executed  
successfully!' } failure { echo 'Pipeline execution  
failed!' } }}
```

Penjelasan

Contoh Jenkinsfile ini menunjukkan struktur dasar pipeline declarative dengan empat tahap: Hello, Build, Test, dan Deploy. Setiap tahap berisi langkah-langkah sederhana untuk mendemonstrasikan sintaks.

Blok post menentukan tindakan yang akan diambil setelah pipeline selesai, tergantung pada hasilnya. Dalam contoh ini, pesan yang berbeda akan ditampilkan untuk keberhasilan dan kegagalan.

Membuat Pipeline **Project Pertama**

Buat Item Baru

Di dashboard Jenkins, klik "New Item", masukkan nama untuk proyek Anda, dan pilih "Pipeline" sebagai jenis proyek. Klik "OK" untuk melanjutkan.

Konfigurasi SCM

Di bagian "Pipeline", pilih "Pipeline script from SCM" jika Jenkinsfile Anda disimpan dalam repository. Pilih jenis SCM (misalnya Git) dan masukkan URL repository.

Tentukan Lokasi Jenkinsfile

Di bidang "Script Path", masukkan lokasi Jenkinsfile relatif terhadap root repository. Nilai default-nya adalah "Jenkinsfile" yang berarti file berada di root.

Simpan Konfigurasi

Klik "Save" untuk menyimpan konfigurasi proyek pipeline Anda. Proyek sekarang siap untuk dijalankan.

Menjalankan dan Memantau Pipeline

Memulai Build

Klik "Build Now" di sidebar proyek untuk memulai eksekusi pipeline. Jenkins akan mengambil kode dari SCM dan menjalankan tahapan yang didefinisikan dalam Jenkinsfile.

Console Output

Klik nomor build dan pilih "Console Output" untuk melihat log eksekusi secara real-time. Ini berguna untuk debugging jika terjadi masalah.

Stage View

Pipeline Stage View memberikan visualisasi grafis dari kemajuan pipeline, menampilkan setiap tahap dengan status dan durasi. Ini memudahkan untuk melihat di mana pipeline berhasil atau gagal.

Troubleshooting Dasar

Error Java/Maven/Gradle Not Found

Pastikan alat yang diperlukan diinstal dan dikonfigurasi dengan benar di Jenkins. Gunakan blok tools dalam Jenkinsfile atau konfigurasi global di Manage Jenkins -> Global Tool Configuration.

Error Git Checkout

Verifikasi URL repository dan kredensial. Pastikan agen Jenkins memiliki akses ke repository. Periksa log untuk pesan error spesifik yang dapat membantu diagnosis.

Plugin Hilang

Jika pipeline gagal karena plugin tidak ditemukan, pergi ke Manage Jenkins -> Manage Plugins untuk menginstal plugin yang diperlukan. Restart Jenkins jika diperlukan.

Jenkinsfile Sintaks Error

Gunakan Pipeline Syntax atau Declarative Pipeline Validator di Jenkins untuk memeriksa sintaks Jenkinsfile Anda sebelum commit.

Tips untuk **Jenkins Pipeline**

Mulai Sederhana

Selalu mulai dengan Jenkinsfile yang sederhana dan tambahkan kompleksitas secara bertahap. Ini memudahkan debugging dan memastikan setiap bagian berfungsi sebelum menambahkan yang baru.

Validasi Sintaks

Gunakan Jenkinsfile Linter atau Declarative Pipeline Validator untuk memeriksa sintaks sebelum commit. Ini dapat menghemat waktu dengan menangkap kesalahan lebih awal.

Snippet Generator

Manfaatkan Snippet Generator di Jenkins untuk membuat langkah-langkah pipeline yang kompleks. Ini membantu dengan sintaks yang benar dan parameter yang diperlukan.

Update Berkala

Jaga Jenkins dan plugin tetap diperbarui untuk mendapatkan fitur terbaru dan perbaikan keamanan. Selalu buat backup sebelum upgrade besar.

Jenkins Agents untuk Build Terdistribusi

Untuk proyek besar atau tim dengan banyak build, pertimbangkan untuk menggunakan Jenkins Agents untuk mendistribusikan beban kerja. Agents memungkinkan Anda menjalankan build di berbagai mesin dengan konfigurasi berbeda, meningkatkan throughput dan memungkinkan pengujian di berbagai lingkungan.

Jenkins Master

Mengelola konfigurasi dan UI

Komunikasi Master-Agent

Protokol JNLP atau SSH

Jenkins Agents

Menjalankan build di berbagai lingkungan



</Rakamin

Terima kasih